

FUNDAÇÃO ESCOLA DE COMÉRCIO ÁLVARES PENTEADO

FECAP

CENTRO UNIVERSITÁRIO ÁLVARES PENTEADO ANÁLISE E
DESENVOLVIMENTO DE SISTEMAS

Ana Julia Bernardo Quintans - 26028857

Clayton Santos Ferreira – 26029419

Leonardo Zachello Encinas - 26028745

Riane Toscano - 26029235

Ricardo Branco Rios - 26029360

PROJETO INTEGRADO
MODELAGEM DE SOFTWARE E ARQUITETURA DE SISTEMAS
PROFESSOR(A) ORIENTADOR(A): AIMAR MARTINS LOPES

São Paulo

2026

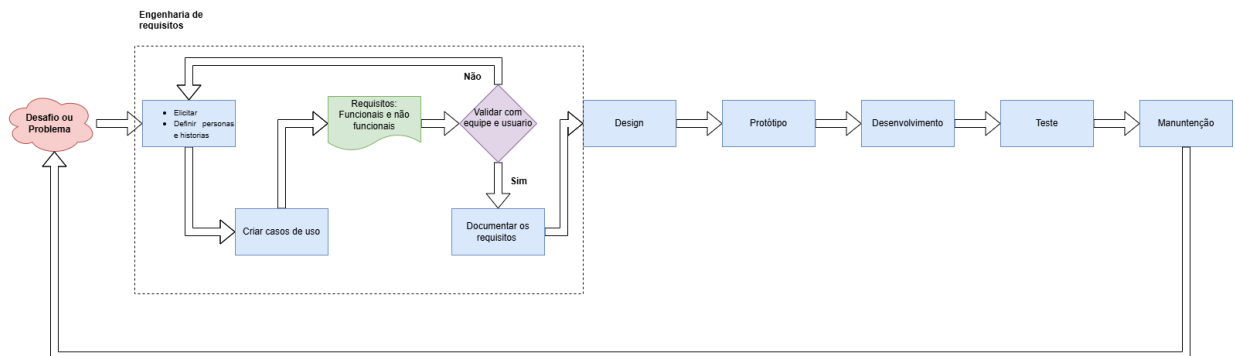
Sumário

Sumário

1 INTRODUÇÃO	3
1.1 – Processo de desenvolvimento de Software	3
1.2 Apresentação	3
2. DESIGN SPRINT – Ideação e prototipação do desafio.....	3
2.1 Desafio.....	3
2.2 Entender Mapear	4
2.2.1 – Entendimento e observação. COMO PODEMOS?	4
2.2.2 – Pesquisa Desk	5
2.3 Ideação – desenho da solução (trilha do usuário)	5
2.4 Prototipagem.....	6
3.REQUISITOS DE SISTEMA	6
3.1 REQUISITOS FUNCIONAIS DE SOFTWARE	6
3.2 REQUISITOS NÃO FUNCIONAIS DE SOFTWARE	8
4. DIAGRAMAS DE UML.....	10
4.1 CASOS DE USO	10
4.2 DIAGRAMA DE CLASSE	14
5. Especificação estrutura de uma rotina do sistema.....	14
5.1 – Apresentar a especificação de acordo com o modelo abaixo.....	14
5.2 – Apresentar a codificação da rotina	16
6 REFERÊNCIAS BIBLIOGRÁFICAS.....	18

1 INTRODUÇÃO

1.1 – Processo de desenvolvimento de Software



1.2 Apresentação

A Messier Data & Creative é uma empresa de desenvolvimento de jogos e experiências interativas que combina tecnologia e criatividade para promover engajamento em eventos, demonstrações de produtos e campanhas de marca. Como novo passo, a Messier pretende lançar uma plataforma de jogos digitais para escolas de ensino fundamental e médio. Neste Projeto Interdisciplinar, as equipes irão desenvolver um aplicativo Desktop para gerenciar assinaturas de jogos educacionais por escolas, controlando pacotes contratados, limites de acessos mensais, IPs autorizados e relatórios de consumo. Fluxo do processo (simulação do módulo Escola): 1) Login: a escola se autentica; o aplicativo verifica se a escola possui pacote ativo e se o IP de origem (informado na simulação) está autorizado. 2) Catálogo: o aplicativo apresenta os games disponíveis no pacote adquirido pela escola. 3) Acesso: a escola simula o acesso a um game; o aplicativo registra e contabiliza o acesso no consumo do mês, respeitando o limite do pacote. Tendo.....

2. DESIGN SPRINT – Ideação e prototipação do desafio

2.1 Desafio

Inicial da proposta do PI - as equipes irão desenvolver um aplicativo Desktop para gerenciar assinaturas de jogos educacionais por escolas, controlando pacotes contratados, limites de acessos mensais, IPs autorizados e relatórios de consumo.

2.2 Entender Mapear

2.2.1 – Entendimento e observação. COMO PODEMOS?

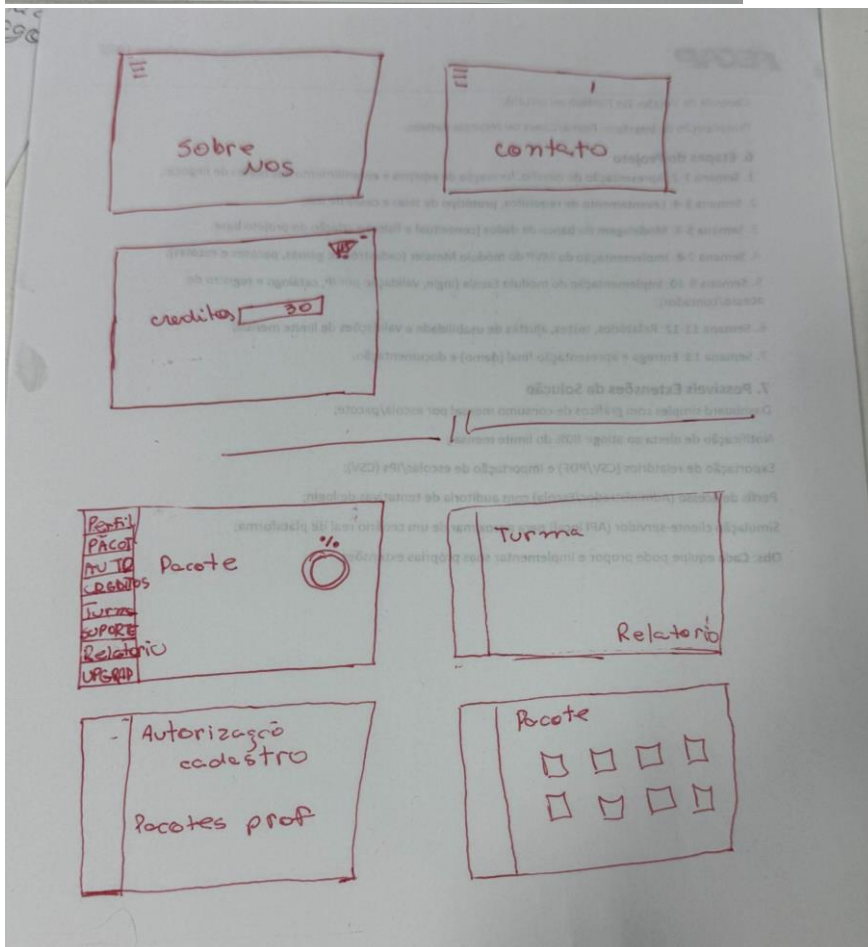
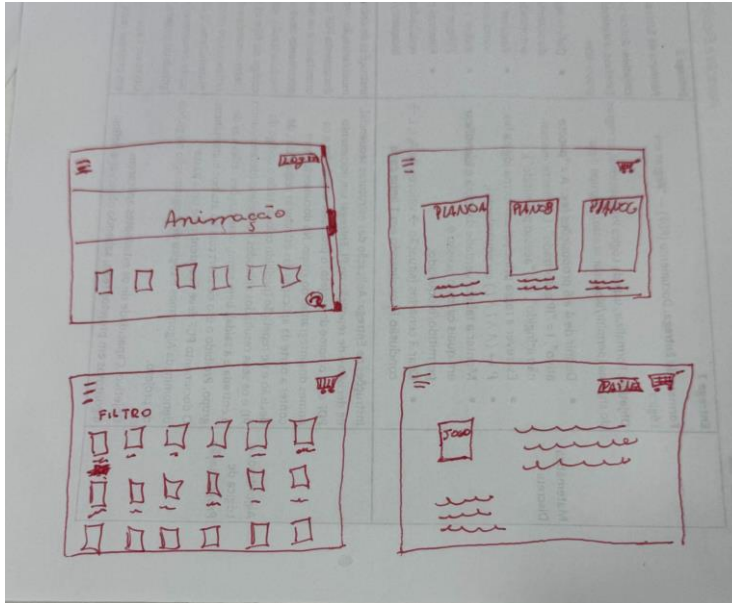
Reuniões frequentes para alinhamento de ideias, planejamento e implementação das ideias.

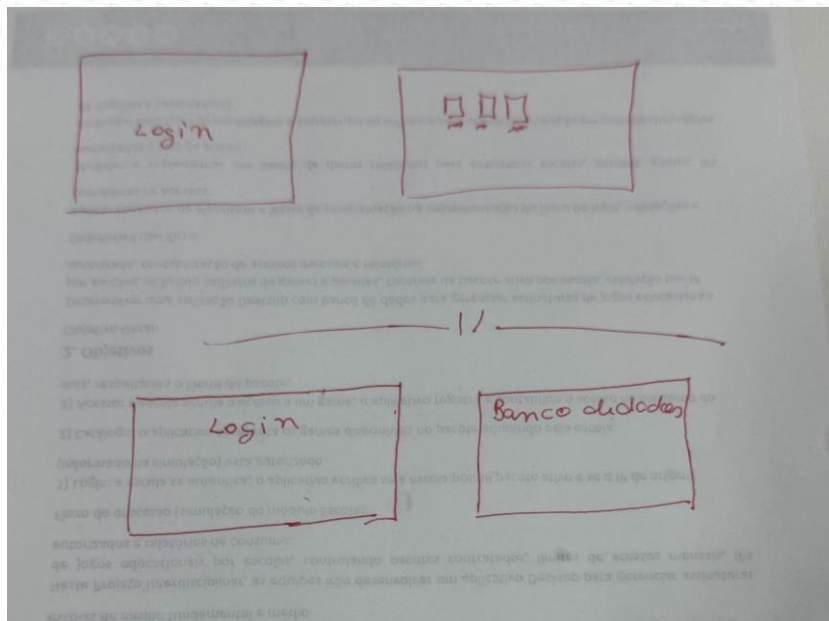
Como podemos?

- Fazer um site atrativo
- Fazer um site auto explicativo
- Fazer Banco de dados
- Como autorizar por IP

2.2.2 – Pesquisa Desk

2.3 Ideação – desenho da solução (trilha do usuário)





2.4 Prototipagem

3.REQUISITOS DE SISTEMA

3.1 REQUISITOS FUNCIONAIS DE SOFTWARE

RFS01	
Função	Cadastro de Games;
Descrição	O sistema deve permitir o cadastro de games (CRUD) com nome, descrição, tema, faixa etária e status;
Entradas	id_game, nome, descricao, tema, faixa_etaria, status;
Fonte	Administrador por meio da interface do sistema;
Saídas	Confirmação de operação, listagem de games, exibição de dados e mensagens de erro;
Ação	O sistema deve permitir cadastrar, consultar, atualizar e excluir games, realizando validações e persistindo os dados em banco.

RFS02	
Função	Cadastro de Pacotes;
Descrição	O sistema deve permitir o cadastro de Pacotes (CRUD) com lista de games, limite de acessos mensais e preço;
Entradas	id_pacote, nome, limite_Acessos_mensal, preco_mensal;
Fonte	Administrador por meio da interface do sistema;

Saídas	Confirmação de operação, listagem de pacotes, exibição de dados e mensagens de erro;
Ação	O sistema deve permitir cadastrar, consultar, atualizar e excluir pacotes, realizando validações e persistindo os dados em banco.

RFS03	
Função	Cadastro de Escolas;
Descrição	O sistema deve permitir o cadastro de escolas (CRUD) com dados básicos, pacote ativo e IPs autorizados;
Entradas	id_escola, nome, login, senha, pacote_ativo, ips_autorizados;
Fonte	Administrador por meio da interface do sistema;
Saídas	Confirmação de operação, listagem de escolas, exibição de dados e mensagens de erro;
Ação	O sistema deve permitir cadastrar, consultar, atualizar e excluir escolas, realizando validações e persistindo os dados em banco.

RFS04	
Função	Login e Validação de Usuário;
Descrição	O sistema deve realizar login da escola, validando pacote ativo e IP de origem;
Entradas	login, senha;
Fonte	Escola por meio da interface do sistema;
Saídas	Acesso concedido à plataforma, acesso negado (com motivo), mensagens de erro/validação, registro de acesso;
Ação	O sistema deve autenticar o usuário, validar credenciais, verificar a existência de pacote ativo dentro da validade e conferir se o acesso está sendo feito a partir de um IP autorizado. Se todas as condições forem atendidas, o acesso é permitido.

RFS05	
Função	Catálogo de Games;
Descrição	O sistema deve exibir o catálogo de games disponíveis conforme o pacote da escola;
Entradas	id_escola, filtros opcionais;

Fonte	Sistema (estado de autenticação) e banco de dados de games;
Saídas	Listagem de games disponíveis conforme seu pacote ativo e filtros (se escolhidos);
Ação	O sistema deve permitir que usuários autenticados tenham acesso ao catálogo de games realizando consulta ao banco de dados e exibindo apenas jogos ativos no pacote do usuário. Deve permitir a utilização de filtros para facilitar a busca.

RFS06	
Função	Registro de Acesso;
Descrição	O sistema deve registrar acessos aos games, contabilizando consumo mensal e bloqueando quando atingir o limite;
Entradas	id_acesso, id_escola, id_game, data e hora, id_ip, limite_acessos_mes;
Fonte	Próprio sistema ao identificar acesso à um game.
Saídas	Registro de acesso, atualização da contagem mensal e liberação ou bloqueio de acesso;
Ação	O sistema deve registrar o acesso ao game e atualizar a contagem mensal para seu respectivo usuário e bloquear o acesso se não tiver mais acessos disponíveis.

3.2 REQUISITOS NÃO FUNCIONAIS DE SOFTWARE

RNFS01	
Função	Modo local;
Descrição	A aplicação deve funcionar offline, para fins de protótipo;
Entradas	Configuração do Sistema, Caminho do Banco de Dados Local;
Fonte	Sistema, Máquina do Usuário;

Saídas	Sistema funcionando sem internet, Dados acessíveis localmente;
Ação	O sistema deve operar de forma offline, utilizando banco de dados local, sem dependência de conexão externa.

RNFS02	
Função	Interface intuitiva;
Descrição	Interface intuitiva e compatível com desktop (Windows), com telas de lista + formulário;
Entradas	Interações do Usuário (Cliques, navegação);
Fonte	Usuário;
Saídas	Navegação simples e compreensível, redução de erros de uso;
Ação	O sistema deve apresentar uma interface clara, organizada e de fácil utilização, permitindo que usuários realizem operações sem necessidade de treinamento avançado.

RNFS03	
Função	Sistema veloz;
Descrição	Tempo de resposta adequado para consultas e relatórios com base de dados pequena/média;
Entradas	Requisições do usuário, Consultas ao banco;
Fonte	Usuário, Sistema;
Saídas	Respostas rápidas, Baixo tempo de carregamento;
Ação	O sistema deve responder às ações do usuário em tempo reduzido, garantindo desempenho adequado mesmo com múltiplos registros.

RNFS04	
Função	Código organizado;
Descrição	Código modular e organizado em camadas (UI, regras de negócio, dados/persistência);
Entradas	Código-fonte do sistema;
Fonte	Desenvolvedor;
Saídas	Código legível e modular;
Ação	O sistema deve ser desenvolvido seguindo boas práticas de programação, com separação em camadas (UI, regras de negócio e persistência), facilitando manutenção e escalabilidade.

RNFS05	
Função	Banco relacional;

Descrição	Persistência em banco relacional com integridade (chaves, relacionamentos e validações);
Entradas	Dados do sistema (usuários, games, acessos, pacotes);
Fonte	Sistema, Usuário;
Saídas	Dados estruturados e consistentes, Integridade relacional;
Ação	O sistema deve utilizar banco de dados relacional, garantindo integridade, consistência e organização dos dados por meio de tabelas e relacionamentos.

RNFS06	
Função	Documentação;
Descrição	Documentação clara e padronizada do projeto e do código (README e manual rápido de execução);
Entradas	Informações do Sistema, códigos e regras de negócio;
Fonte	Desenvolvedor;
Saídas	Documentação técnica, Documentação de Uso;
Ação	O sistema deve possuir documentação clara, contendo descrição dos requisitos, estrutura do banco de dados, regras de negócio e instruções de uso.

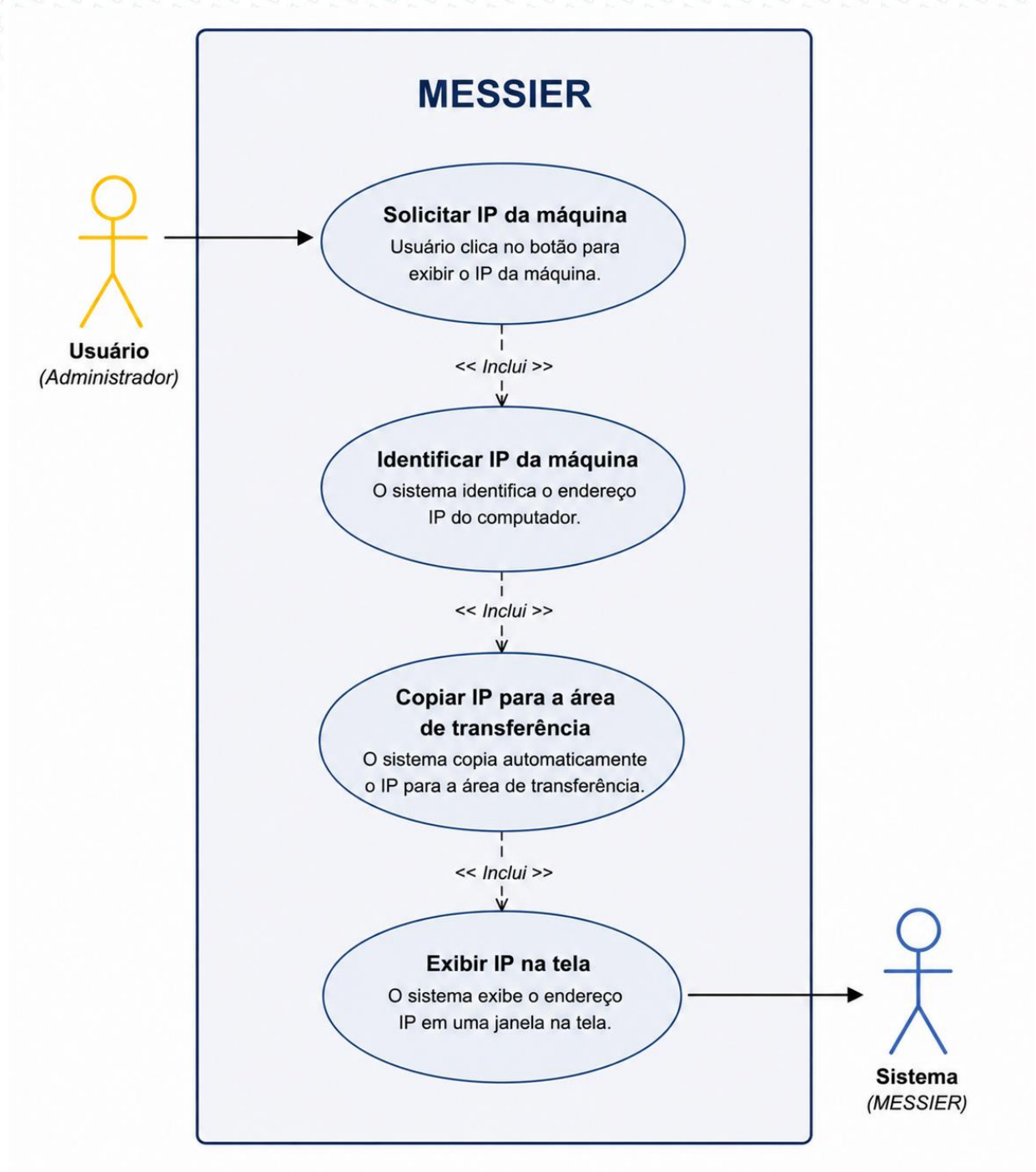
4. DIAGRAMAS DE UML

4.1 CASOS DE USO

Caso de Uso 1 — Consulta do IP da máquina

O usuário clica no botão responsável por exibir o IP da máquina.

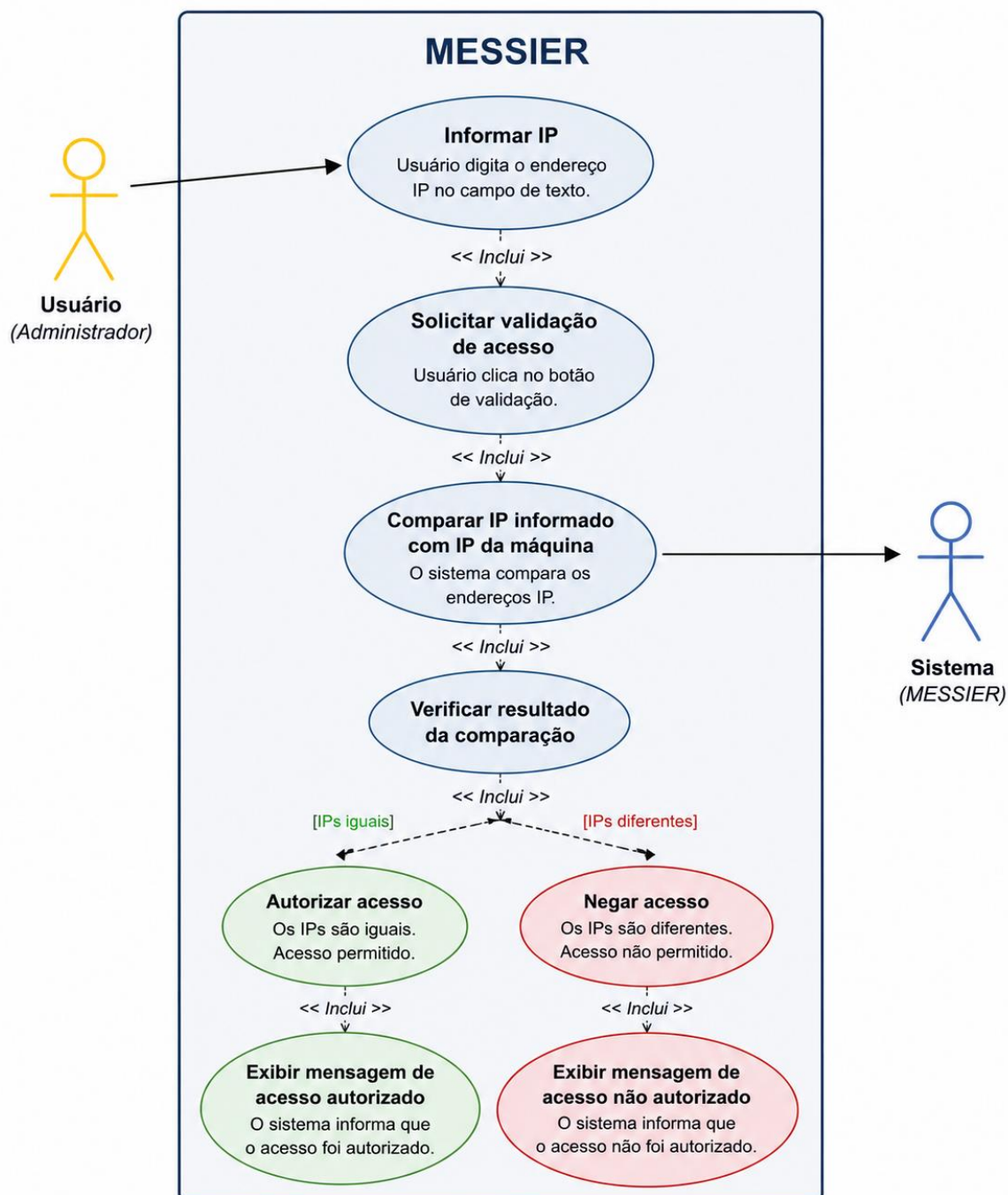
O sistema identifica o endereço IP do computador, copia automaticamente o valor para a área de transferência e exibe a informação em uma janela na tela.



Caso de Uso 2 — Validação de acesso por IP

O usuário digita um endereço IP no campo de texto e clica no botão de validação.

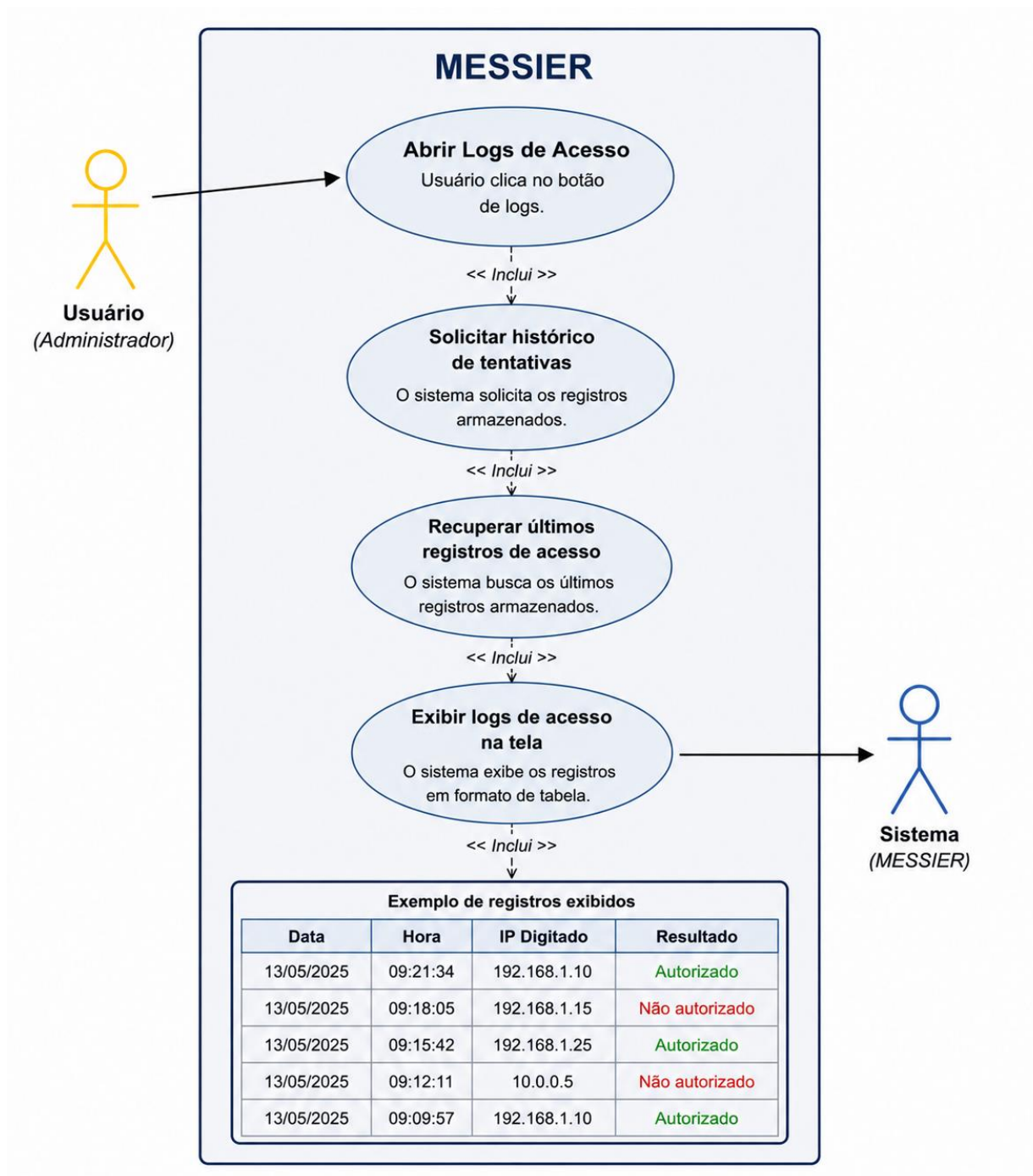
O sistema compara o IP informado com o IP da máquina. Caso os valores sejam iguais, o acesso é autorizado. Caso contrário, o sistema informa que o acesso não é autorizado.



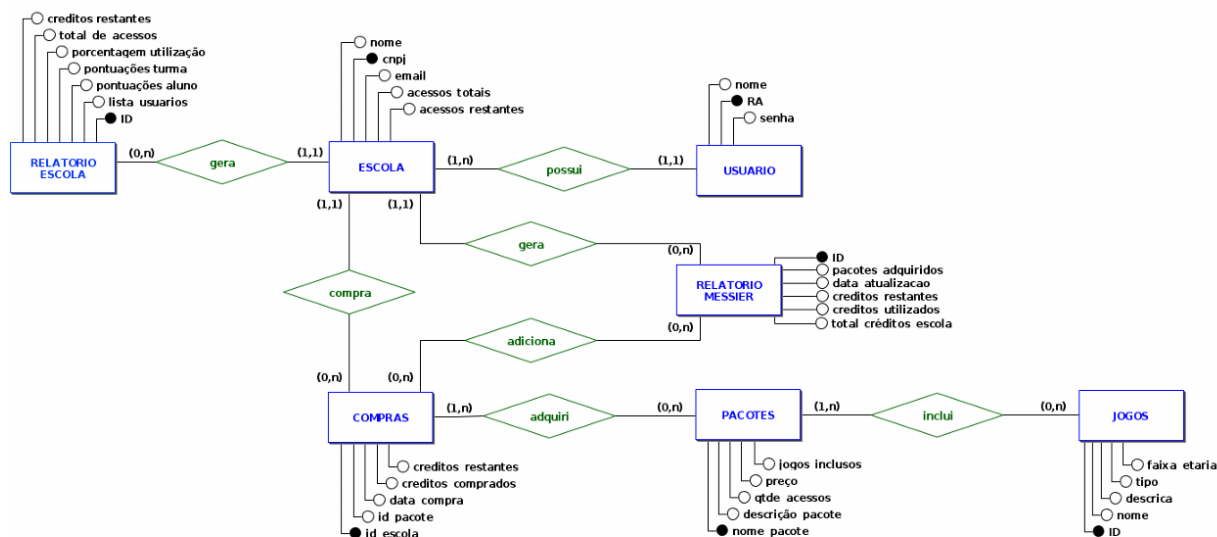
Caso de Uso 3 — Consulta de logs de acesso

O usuário clica no botão de logs para visualizar o histórico de tentativas de acesso realizadas no sistema.

O programa exibe os últimos registros armazenados, mostrando data, horário, IP digitado e o resultado da validação.



4.2 DIAGRAMA DE CLASSE



5. Especificação estrutura de uma rotina do sistema

5.1 – Apresentar a especificação de acordo com o modelo abaixo

Sistema/Software:

Messier — Plataforma de Gerenciamento de Jogos Educacionais

Função:

Consulta e identificação automática do endereço IP da máquina utilizada pela escola durante o processo de autenticação e validação de acesso.

Descrição:

O usuário acessa o módulo da escola e clica no botão responsável por consultar o IP da máquina. O sistema Messier realiza a leitura do endereço IP ativo do computador conectado à rede, verifica se o IP foi obtido corretamente, copia automaticamente o valor para a área de transferência do sistema operacional e exibe o endereço identificado em uma janela de confirmação na tela.

O processo permite que o usuário visualize rapidamente o IP utilizado para autenticação e validação de acesso na plataforma

Entradas:

- Clique no botão “Consultar IP”
- Solicitação de leitura da interface de rede da máquina
- Dados da conexão de rede ativa do computador

Fonte:

- Usuário (Administrador ou Escola)
- Adaptador de rede do sistema operacional

Saídas:

- Endereço IP identificado
- Mensagem de confirmação
- IP copiado automaticamente para a área de transferência
- Registro da operação no sistema

Destino:

- Janela de exibição do sistema Messier
- Área de transferência do Windows
- Memória temporária da aplicação

Ações:

- O usuário clica no botão “Consultar IP”
- O sistema verifica se existe conexão de rede ativa
- O programa acessa os adaptadores de rede disponíveis no computador
- O sistema identifica o endereço IPv4 ativo da máquina
- O programa valida se o IP encontrado é válido
- O sistema converte o endereço IP para formato de texto
- O IP é armazenado temporariamente na aplicação
- O sistema copia automaticamente o IP para a área de transferência
- O programa abre uma janela informando o endereço IP encontrado
- O sistema exibe mensagem de confirmação de cópia
- O usuário pode utilizar o IP para autenticação ou cadastro de IP autorizado

Requisitos:

- Aplicação Messier em execução
- Computador conectado à rede
- Interface de rede habilitada
- Permissão de acesso às informações de rede do sistema operacional
- Área de transferência do Windows habilitada
- Biblioteca de rede configurada no projeto C#

Pré-condição:

- O sistema Messier deve estar aberto
- O usuário deve possuir acesso ao módulo da escola
- O computador deve possuir conexão de rede ativa

Pós-condições:

- O endereço IP da máquina é identificado corretamente
- O IP é exibido em tela
- O IP é copiado automaticamente
- O usuário pode reutilizar o IP em processos de autenticação e validação

Efeitos colaterais

- O conteúdo anterior da área de transferência será substituído
- O sistema poderá exibir mensagem de erro caso não exista conexão ativa
- O IP exibido pode mudar dependendo da rede utilizada pelo computador
- O sistema poderá registrar a consulta do IP em logs internos da aplicação

5.2 – Apresentar a codificação da rotina**INÍCIO**

Quando o usuário clicar no botão "Consultar IP":

Verificar se o computador possui conexão de rede

Se não houver conexão:

Exibir mensagem "Nenhuma conexão de rede encontrada"

Encerrar processo

Buscar o endereço IP da máquina

Se o IP for encontrado:

Copiar o IP para a área de transferência

Exibir mensagem:

"IP da máquina: [IP encontrado]"

"IP copiado para a área de transferência"

Senão:

Exibir mensagem "Não foi possível identificar o IP da máquina"

FIM

</> Python

```
import socket
import pyperclip

def consultar_ip():
    try:
        nome_maquina = socket.gethostname()
        ip_maquina = socket.gethostbyname(nome_maquina)

        if ip_maquina:
            pyperclip.copy(ip_maquina)
            print("IP da máquina:", ip_maquina)
            print("IP copiado para a área de transferência.")
        else:
            print("Não foi possível identificar o IP da máquina.")

    except:
        print("Erro ao consultar o IP da máquina.")

consultar_ip()
```


6 REFERÊNCIAS BIBLIOGRÁFICAS

SOMMERVILLE, Ian. **Engenharia de software**. 11. ed. São Paulo: Pearson Addison-Wesley, 2017.

PYTHON SOFTWARE FOUNDATION. **Documentação oficial do Python**. Disponível em: [Python.org](https://python.org). Acesso em: 11 maio 2026.

PYTHON SOFTWARE FOUNDATION. **Socket — Python Documentation**. Disponível em: [Python Socket Documentation](https://pythonsocket.com). Acesso em: 11 maio 2026.

PYPERCLIP. **Documentation — Pyperclip**. Disponível em: [Pyperclip Documentation](https://pyperclip.com). Acesso em: 11 maio 2026.

PRESSMAN, Roger S. **Engenharia de software: uma abordagem profissional**. 8. ed. Porto Alegre: AMGH, 2016.

SOMMERVILLE, Ian. **Engenharia de software**. 10. ed. São Paulo: Pearson, 2019.

STALLINGS, William. **Redes e sistemas de comunicação de dados**. São Paulo: Pearson, 2017.